

Participation Activity 10

Last Name: _____

First Name: _____

Std #: _____

Solution

- In our Lecture 36, we unrolled our loop using 2 accumulators -> **total1** and **total2**, hence creating two sequences of independent additions:

```
int sum_2x2unrolled(int *A, int n) {  
    int total1 = 0;  
    int total2 = 0;  
    int granTotal = 0;  
    int i;  
  
    for (i = 0; i < n-1; i += 2) {  
        total1 += A[i];  
        total2 += A[i+1];  
    }  
    granTotal = total1 + total2;  
    for (; i < n; i++)  
        granTotal += A[i];  
  
    return granTotal;  
}
```

total1 is stored
in the register **%eax**
and **total2** is stored
in the register **%ecx**

```
loop:  
    # total1 += A[i]  
    addl (%rdx), %eax  
    # total2 += A[i+1]  
    addl 4(%rdx), %ecx  
    addq $8, %rdx  
    cmpq %r8, %rdx  
    jne  loop
```

Participation Activity 10

Last Name: _____

First Name: _____

Std #: _____

Solution

- Imagine we tweak our summing function such that we create the following two functions:

```
sum_10x10unrolled(int *A, int N);
```

If we use 10 accumulators, we need 10 (integer) registers, which we have in x86-64.

```
sum_20x20unrolled(int *A, int N);
```

Now, if we use 20 accumulators, we need 20 (integer) registers, which we do not have in x86-64.

Participation Activity 10

Last Name: _____

First Name: _____

Std #: _____

Solution

- After benchmarking them, we discover that
`sum_20x20unrolled(int *A, int N);` is slower than
`sum_10x10unrolled(int *A, int N);`
- Can you explain what may be happening? *Hint: Think registers!*
Since we do not have 20 (integer) registers in x86-64, then memory locations (such as location in stack – this is called “spilling”) would be used to hold the accumulators that have not been assigned to a register. Since memory access (e.g., stack access) is slower than register access, then our function
`sum_20x20unrolled(int *A, int N);` would be slower than
our function `sum_10x10unrolled(int *A, int N);`

There
may
also be
other
reasons!